

Set & Queue Interfaces Deep Dive

Complete Conceptual, Architectural, Internal & Interview Guide • HashSet, LinkedHashSet, TreeSet, ArrayDeque, PriorityQueue

This masterclass covers the **Set** and **Queue** families of the Java Collections Framework (JCF). Rather than memorizing methods, this guide builds a rigorous understanding of **hash tables**, **bucket chaining**, **tree balanced lookups**, **binary min/max heaps**, **circular ring buffers**, **contracts**, and **algorithmic traps**.

PART 1: The Set Interface

1. What is a Set & Why Does It Exist?

A **Set** models the mathematical abstraction of a set: a collection of elements containing **no duplicates**. While a `List` models an ordered, indexed sequence where elements may repeat, a `Set` models membership and uniqueness.

Core Properties:

- **Uniqueness:** Enforces that no two elements e_1, e_2 satisfy $(e_1 == \text{ext}\{\text{null}\} \text{ ext}\{ ? \} e_2 == \text{ext}\{\text{null}\} : e_1. \text{ext}\{\text{equals}\}(e_2))$.
- **No Positional Indexing:** Unlike `List.get(int index)`, a `Set` has no index-based retrieval. Elements are accessed via iterators or membership queries (`contains()`).
- **Duplicate Determination:** In Java, duplicate checking is **never** based on memory reference equality (`==`) alone; it is determined by the combination of `hashCode()` and `equals()` in hash sets, or `compareTo()` / `compare()` in tree sets.

2. Set Interface Hierarchy

```
Iterable<T> (java.lang) | ↳ Collection<E> (java.util) | ↳ Set<E> (Uniqueness contract) | ↳ HashSet<E>
(Backing: HashMap, O(1) avg, unordered) | | ↳ LinkedHashSet<E> (Backing: LinkedHashMap, insertion-ordered) |
↳ SortedSet<E> (Total ordering contract: first(), last(), subSet()) | ↳ NavigableSet<E> (Closest-match
navigation: lower(), floor(), ceiling(), higher()) | ↳ TreeSet<E> (Backing: Red-Black TreeMap, O(log N),
sorted)
```

3. HashSet: Internal Architecture & Mechanics

An `HashSet<E>` does **not** implement hashing from scratch. Internally, it is backed by a `HashMap<E, Object>` instance.

```
public class HashSet<E> extends AbstractSet<E> implements Set<E> {
    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object(); // Dummy value

    public boolean add(E e) {
        return map.put(e, PRESENT) == null; // put returns null if key was new
    }
}
```

Step-by-Step Internal Execution of `set.add("A")`: 1. Calculate hash: `int h = "A".hashCode() ^ (h >>> 16)` 2. Compute bucket index: `i = (table.length - 1) & h` 3. Traverse bucket chain at index `i`: - If a Node matches: `(node.hash == h && (node.key == "A" || "A".equals(node.key)))` -> Key already exists! Return false (no insertion). - If end of chain reached with no match: -> Insert new Node("A", PRESENT) into bucket. Return true.

4. Custom Objects in HashSet: The equals() and hashCode() Contract

When using custom classes as elements in a `HashSet`, failure to override both `hashCode()` and `equals()` breaks duplicate prevention completely.

⚠ The Fundamental Hash Contract

1. If `a.equals(b) == true`, then `a.hashCode() == b.hashCode()` **MUST be true**.
2. If `a.hashCode() == b.hashCode()`, `a.equals(b)` **is NOT necessarily true (Hash Collision)**.
 - If only `equals()` is overridden: Two logical duplicates yield different `hashCode()` values, landing in different buckets → Duplicate inserted!
 - If only `hashCode()` is overridden: Two objects land in the same bucket, but default `equals()` (memory identity `==`) fails → Duplicate inserted!

```

public class Student {
    private final int id;
    private final String name;

    public Student(int id, String name) { this.id = id; this.name = name; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return id == student.id && Objects.equals(name, student.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}

```

5. Mutable Objects in HashSet (Critical Bug Pattern)

If an object's field used in `hashCode()` or `equals()` is modified after insertion into a `HashSet`, its computed hash changes. Subsequent calls to `set.contains(obj)` or `set.remove(obj)` compute a new bucket index, searching the wrong bucket and returning `false`. **Rule:** Never mutate objects acting as hash set elements or map keys!

6. LinkedHashSet: Insertion-Order Preservation

`LinkedHashSet<E>` extends `HashSet<E>` and is backed by a `LinkedHashMap`. It maintains a **doubly linked list running across all its bucket entries**.

- **Ordering Semantics:** Preserves **insertion order** (or access order if configured in map), NOT sorted order.
- **Performance & Cost:** Read/Write complexity remains $O(1)$. Iteration is faster than `HashSet` ($O(\text{ext}\{\text{size}\})$) instead of $O(\text{ext}\{\text{capacity}\})$, but node pointer overhead is higher (2 extra pointers per entry).

7. TreeSet: Balanced Search Tree & Red-Black Invariants

`TreeSet<E>` implements `NavigableSet<E>` and is backed internally by a balanced **Red-Black TreeMap**.

- **Complexity:** Guarantees $O(\log N)$ time for `add()`, `remove()`, and `contains()`.
- **Ordering:** Elements are sorted according to their **Natural Ordering** (via `Comparable`) or a provided `Comparator`.
- **Null Handling:** Rejects `null` elements in natural ordering (throws `NullPointerException` on `compareTo()` call).

⚠️ TreeSet Uniqueness Trap (compareTo vs equals)

`TreeSet` determines uniqueness **strictly via** `compareTo() == 0` **or** `comparator.compare() == 0`, ignoring `equals()`!

If two objects have different names but equal IDs, and your `compareTo()` compares only IDs, `TreeSet` treats them as duplicates and discards the second object, even though `equals()` returns `false`.

8. Set Implementations Comparison & Decision Matrix

Feature	HashSet	LinkedHashSet	TreeSet
Backing Structure	HashMap (Hash Table + Buckets)	LinkedHashMap (Hash Table + Doubly-linked chain)	TreeMap (Red-Black Balanced Tree)
Element Ordering	Unordered / Unpredictable	Strict Insertion Order	Sorted (Natural or Comparator)
Duplicate Detection	<code>hashCode()</code> + <code>equals()</code>	<code>hashCode()</code> + <code>equals()</code>	<code>compareTo() == 0</code> or <code>compare() == 0</code>
Null Support	Allows 1 <code>null</code>	Allows 1 <code>null</code>	No <code>null</code> (Throws NPE)
Time Complexity (Avg)	$O(1)$ add / contains / remove	$O(1)$ add / contains / remove	$O(\log N)$ all operations
Memory Consumption	Low	Moderate (Extra pointers)	High (Tree node overhead)

9. What is a Queue? (FIFO Architecture)

10. The Queue Methods Dual-Contract Trap

Operation Type	Throws Exception (Unchecked)	Returns Special Value (Safe)	Failure Condition
Insert (Tail)	<code>add(e)</code> → <code>IllegalStateException</code>	<code>offer(e)</code> → Returns <code>false</code>	Capacity-restricted queue is full
Remove (Head)	<code>remove()</code> → <code>NoSuchElementException</code>	<code>poll()</code> → Returns <code>null</code>	Queue is completely empty
Examine (Head)	<code>element()</code> → <code>NoSuchElementException</code>	<code>peek()</code> → Returns <code>null</code>	Queue is completely empty

`ArrayDeque<E>` implements `Deque<E>` using a resizable circular array buffer with `head` and `tail` pointer indices.

- **Circular Math:** Buffer wrapping uses fast bitwise masking: `head = (head - 1) & (elements.length - 1)`. Array sizes are strictly maintained as powers of 2 (2^k).
- **Null Policy:** Rejects `null` elements. `null` is reserved as a sentinel return value for `poll()` and `peek()`.
- **Why ArrayDeque Beats Stack & LinkedList:** Zero synchronization lock overhead, no node heap allocation churn, and outstanding CPU spatial cache locality.

The `Deque<E>` (Double-Ended Queue) interface extends `Queue<E>` to provide operations at both endpoints, naturally unifying FIFO and LIFO structures.

```
// Modern Standard Stack Implementation
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10); // inserts at front (head)
stack.push(20);
int top = stack.pop(); // removes from front (head) -> 20
```

A `PriorityQueue<E>` is an unbounded priority queue based on a **complete binary heap** stored contiguously in an `Object[]` queue array.

Heap Operations & Complexity

- Page 3 of 12

⚠️ PriorityQueue Interview Traps

1. **Iteration is NOT Sorted:** Iterating over a `PriorityQueue` via `for-each` traverses the backing array in level-order heap storage, *not* sorted order! Only successive calls to `poll()` yield sorted order.
2. **Min-Heap vs. Max-Heap:** Default is Min-Heap (smallest at root). For Max-Heap, instantiate with `Collections.reverseOrder()`:
`PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());`

PART 3: Complete Unified Complexity Reference

Collection	Add / Offer	Remove / Poll	Contains / Search	Peek / Access	Underlying Data Structure
HashSet	$O(1)$ avg / $O(N)$ worst	$O(1)$ avg / $O(N)$ worst	$O(1)$ avg / $O(N)$ worst	N/A	HashMap (Hash table)
LinkedHashSet	$O(1)$ avg / $O(N)$ worst	$O(1)$ avg / $O(N)$ worst	$O(1)$ avg / $O(N)$ worst	N/A	LinkedHashMap (Hash table + Doubly-linked list)
TreeSet	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(1)$ (first/last)	TreeMap (Red-Black Balanced Tree)
ArrayDeque	Amortized $O(1)$	Amortized $O(1)$	$O(N)$	$O(1)$	Resizable Circular Array Buffer
LinkedList (Queue)	$O(1)$	$O(1)$	$O(N)$	$O(1)$	Doubly-Linked List
PriorityQueue	$O(\log N)$	$O(\log N)$	$O(N)$ (Linear scan)	$O(1)$	Binary Heap (Array-backed)

PART 4: Implementation from Scratch (Interview Mechanics)

1. Simplified HashSet from Scratch

```
public class MyHashSet<K> {
    private static class Node<K> {
        K key; Node<K> next;
        Node(K key, Node<K> next) { this.key = key; this.next = next; }
    }
    private Node<K>[] buckets = new Node[16];
    private int size = 0;

    private int getBucketIndex(K key) {
        return key == null ? 0 : Math.abs(key.hashCode()) % buckets.length;
    }

    public boolean add(K key) {
        int idx = getBucketIndex(key);
        for (Node<K> curr = buckets[idx]; curr != null; curr = curr.next) {
            if (Objects.equals(curr.key, key)) return false; // Duplicate found
        }
        buckets[idx] = new Node<>(key, buckets[idx]);
        size++;
        return true;
    }

    public boolean contains(K key) {
        int idx = getBucketIndex(key);
        for (Node<K> curr = buckets[idx]; curr != null; curr = curr.next) {
            if (Objects.equals(curr.key, key)) return true;
        }
        return false;
    }
}
```

2. Simplified Circular ArrayDeque from Scratch

```
public class MyArrayDeque<E> {
    private Object[] data = new Object[8];
    private int head = 0, tail = 0;

    public void addLast(E e) {
        if (e == null) throw new NullPointerException();
        data[tail] = e;
        tail = (tail + 1) & (data.length - 1);
    }

    public void addFirst(E e) {
        if (e == null) throw new NullPointerException();
        head = (head - 1) & (data.length - 1);
        data[head] = e;
    }

    @SuppressWarnings("unchecked")
    public E pollFirst() {
        if (head == tail) return null;
        E res = (E) data[head];
        data[head] = null;
        head = (head + 1) & (data.length - 1);
        return res;
    }
}
```

PART 5: 100 High-Yield Interview Questions & Answers

Level 1: Fundamentals (Q1 – Q20)

1. What is the key functional difference between List and Set?

`List` maintains an ordered, indexed sequence that allows duplicates. `Set` models mathematical set uniqueness, forbidding duplicates with no positional indexing.

2. Does Set allow null elements?

`HashSet` and `LinkedHashSet` allow a single `null` element. `TreeSet` forbids `null` when using natural ordering.

3. What is the root interface of Queue?

`Collection<E>`, which in turn extends `Iterable<T>`.

4. Why does queue offer() exist alongside add()?

`add()` throws an unchecked `IllegalStateException` when a capacity-bounded queue is full; `offer()` safely returns `false`.

5. What is the difference between poll() and remove()?

`remove()` throws `NoSuchElementException` on an empty queue; `poll()` returns `null` safely.

6. What is the difference between peek() and element()?

`element()` throws `NoSuchElementException` if empty; `peek()` returns `null`.

7. Is HashSet synchronized/thread-safe?

No. External synchronization via `Collections.synchronizedSet()` or concurrent sets like `ConcurrentHashMap.newKeySet()` is required.

8. What is the default initial capacity and load factor of HashSet?

Initial capacity is 16; load factor is 0.75 (inherited from the underlying `HashMap`).

9. What ordering does LinkedHashSet guarantee?

Strict insertion order (the sequence in which elements were added).

10. What ordering does TreeSet guarantee?

Sorted ascending order according to natural ordering (`Comparable`) or a custom `Comparator`.

11. What is the time complexity of TreeSet add()?

$O(\log N)$ due to balanced Red-Black tree traversal.

12. What data structure backs ArrayDeque?

A resizable circular array buffer with power-of-two capacity.

13. Does PriorityQueue guarantee FIFO ordering?

No. It orders elements based on priority (natural min-heap by default), not arrival time.

14. What is the time complexity of PriorityQueue peek()?

$O(1)$ because the minimum/maximum element is always at the root (index 0).

15. What is the time complexity of PriorityQueue poll()?

$O(\log N)$ due to the downward heapify (siftDown) operation.

16. Can you insert null into ArrayDeque?

No. ArrayDeque rejects `null` because `null` is used as a sentinel return value by `poll()` and `peek()`.

17. Can you insert null into PriorityQueue?

No. PriorityQueue needs to compare elements to maintain heap order; calling `compareTo(null)` triggers `NullPointerException`.

18. What interface represents a double-ended queue in Java?

`java.util.Deque<E>`.

19. How do you instantiate a Max-Heap using PriorityQueue?

`new PriorityQueue<>(Collections.reverseOrder())` or `new PriorityQueue<>((a, b) -> b - a)`.

20. What is NavigableSet?

A sub-interface of `SortedSet` providing closest-match search methods (`ceiling()`, `floor()`, `higher()`, `lower()`).

Level 2: Deep Understanding (Q21 – Q40)**21. How does HashSet store values internally?**

As keys in a backing `HashMap`, associating each element with a static dummy `PRESENT` Object.

22. Why does HashSet use a dummy object instead of null as the map value?

Because `map.put(key, value)` returns `null` when the key is brand new. If `null` were stored, `put()` returning `null` would be ambiguous.

23. If two objects return the same hashCode, are they equal?

No. This is a hash collision. They are placed in the same bucket, where `equals()` distinguishes them.

24. If two objects are equal via equals(), can they have different hashCodes?

No. That violates the Java Object Contract and breaks hash-based collections completely.

25. How does TreeSet decide whether an element is duplicate?

Using `compareTo() == 0` or `comparator.compare() == 0`. It completely bypasses `equals()`.

26. Why does iterating over PriorityQueue not print elements in sorted order?

Because iteration traverses the backing array in breadth-first heap storage order, not sorted order. Only successive `poll()` calls yield sorted order.

27. Why is ArrayDeque preferred over Stack for LIFO operations?

`Stack` extends `Vector` and incurs heavy synchronization lock overhead. `ArrayDeque` is unsynchronized and faster.

28. How does ArrayDeque achieve circular indexing without expensive modulo (%) operations?

By keeping array capacity a power of 2 and using bitwise masking: `(index + 1) & (length - 1)`.

29. Why does LinkedList implement both List and Deque?

Its doubly-linked node structure provides natural $O(1)$ operations at both ends, fitting Deque semantics.

30. When should you choose LinkedList over ArrayDeque for Queue operations?

Almost never. `ArrayDeque` has better cache locality and lower memory footprint. `LinkedList` is only justified when strictly avoiding array resizing pauses.

31. What is the time complexity of searching an arbitrary element in PriorityQueue?

$O(N)$ because the binary heap is not fully sorted like a BST; it requires a linear array scan.

32. What happens if you add a mutable object to a HashSet and then modify its fields?

Its hash code changes, stranding the object in the wrong bucket. `contains()` and `remove()` will fail.

33. What is the difference between Comparable and Comparator?

`Comparable` defines an object's natural ordering via `compareTo()`. `Comparator` defines external custom ordering strategies via `compare()`.

34. How does LinkedHashSet maintain insertion order internally?

Each bucket node has `before` and `after` pointers forming a doubly linked list across all entries.

35. What is the time complexity of NavigableSet ceiling() and floor()?

$O(\log N)$ by traversing the balanced Red-Black tree.

36. What is the load factor in HashSet?

A threshold (default 0.75) that determines when the backing hash table capacity doubles to prevent long collision chains.

37. Why is TreeSet memory consumption higher than HashSet?

Each `TreeMap.Entry` allocates parent, left, right pointers, a boolean color flag, key, and value references.

38. How does PriorityQueue dynamically resize?

If capacity < 64, it grows by 100% (+2); otherwise, it grows by 50% using `Arrays.copyOf`.

39. What is the difference between Deque push() and offerFirst()?

`push()` throws `IllegalStateException` if capacity is exceeded; `offerFirst()` returns `false`.

40. Can a custom class be used in TreeSet without implementing Comparable?

Yes, provided an explicit `Comparator` is passed to the `TreeSet` constructor.

Level 3: Internal Mechanics & Architecture (Q41 – Q60)

41. How does HashMap handle high collision degradation in Java 8+?

When a bucket chain reaches 8 nodes and table size ≥ 64 , it treeifies into a Red-Black Tree ($O(\log N)$).

42. How does siftUp work during PriorityQueue offer()?

The new element is placed at index k , compared with its parent at $(k-1)/2$, and swapped upward until the heap invariant is satisfied.

43. How does siftDown work during PriorityQueue poll()?

The root is replaced with the last element and compared against the smaller child; it is swapped downward until heap order is restored.

44. Why must hash table capacities strictly be powers of two in HashSet/HashMap?

It allows computing bucket indices using fast bitwise AND: `hash & (capacity - 1)` instead of slow modulo arithmetic (`%`).

45. What is the purpose of the bit-shift perturbation hash function in HashSet?

`h ^ (h >>> 16)` spreads higher bits into lower bits, reducing collisions when the table length is small.

46. Why does ArrayDeque double its capacity when head == tail?

Because `head == tail` signifies that the circular buffer is completely full.

47. What is the difference between Red-Black Tree in TreeSet and AVL Tree?

Red-Black trees allow slightly more relaxed balancing than AVL trees, requiring fewer tree rotations during insertions and deletions.

48. Why does TreeSet implement NavigableSet instead of just SortedSet?

`NavigableSet` extends `SortedSet` with directional search (`floor`, `ceiling`, `descendingSet`).

49. What happens internally when you pass an initial capacity to HashSet?

It instantiates a backing `HashMap` sized to `Math.max((int)(cap / 0.75f) + 1, 16)`.

50. How does the equals contract define transitivity?

If `a.equals(b)` is true and `b.equals(c)` is true, then `a.equals(c)` must be true.

51. Why is ArrayDeque memory-efficient compared to LinkedList?

It allocates a single contiguous object reference array rather than separate Node objects for every element.

52. How does PriorityQueue construct a heap from an unsorted collection in O(N) time?

Using Floyd's Bottom-Up Heapify algorithm, applying `siftDown` starting from the last non-leaf parent down to index 0.

53. What is the time complexity of building a PriorityQueue by calling offer() N times?

$O(N \log N)$, which is slower than heapifying a pre-populated array ($O(N)$).

54. What is the memory footprint of an empty HashSet instance?

Approximately 32 bytes for the HashSet wrapper plus 48 bytes for the backing HashMap instance (~80 bytes minimum).

55. Why is LinkedHashSet iteration O(N) while HashSet iteration is O(N + Capacity)?

`LinkedHashSet` traverses its internal linked list directly; `HashSet` must scan every bucket array slot including empty ones.

56. How does subSet() in TreeSet work without copying elements?

It returns a bounded `NavigableSubSet` view backed directly by the parent Red-Black tree.

57. What happens if a Comparator violates the consistency contract (e.g., non-transitive)?

TreeSet can behave unpredictably, losing existing elements or duplicating entries.

58. Why does PriorityQueue heap array indexing start at 0 instead of 1?

To maximize array slot utilization. Parent is $(i-1)/2$, left child is $2i+1$, right child is $2i+2$.

59. Can you change the priority of an element inside PriorityQueue in O(log N)?

No. Standard `PriorityQueue` does not maintain an index map; locating the item requires $O(N)$ before $O(\log N)$ heapify.

60. What is the fail-fast mechanism in HashSet iterators?

The iterator checks `modCount`; if structural changes happen during iteration outside the iterator, it throws `ConcurrentModificationException`.

Level 4: Tricky Traps & Corner Cases (Q61 – Q80)

61. What happens when HashSet receives a custom object with a hardcoded hashCode() returning 1?

Every element lands in bucket 1. The HashSet degrades into a linked list/tree with $O(N)$ or $O(\log N)$ time complexity.

62. Why does TreeSet allow removing an element using an object that is not equal via equals()?

Because `TreeSet` relies solely on `compareTo() == 0` or `compare() == 0` to locate and remove elements.

63. Can you store multiple nulls in LinkedHashSet?

No. Like `HashSet`, only a single `null` element is permitted.

64. What happens when two elements compareTo() to 0 in TreeSet?

The set considers them identical duplicates; the second element is rejected, and `add()` returns `false`.

65. Why is ArrayDeque faster than java.util.Stack even in single-threaded code?

`Stack` synchronizes every call on the object monitor lock, incurring synchronization overhead. `ArrayDeque` has zero locking overhead.

66. Why does PriorityQueue remove(Object) take O(N) time?

It must linearly scan the backing array ($O(N)$) to find the element before executing $O(\log N)$ siftDown.

67. If a class overrides equals() but not hashCode(), how does HashSet behave?

Duplicate instances receive different default system hash codes, land in different buckets, and get added as duplicates.

68. What happens if a Comparator subtraction (a - b) overflows with Integer.MIN_VALUE?

Integer underflow occurs, producing an incorrect sign and corrupting the sort order. Use `Integer.compare(a, b)` instead.

69. Can you create an unmodifiable Set in modern Java (Java 9+)?

Yes, using `Set.of(e1, e2, ...)` or `Set.copyOf(existingSet)`. These reject `null` elements.

70. Why does Set.of("A", "A") throw IllegalArgumentException immediately at runtime?

Java 9+ immutable factory methods explicitly detect duplicate arguments during construction to prevent programming bugs.

71. Is PriorityQueue stable (preserving FIFO order among equal-priority items)?

No. `PriorityQueue` makes no guarantees regarding the relative ordering of ties.

72. How do you implement a Monotonic Queue in Java?

Using an `ArrayDeque` to maintain elements in strictly ascending or descending order by polling smaller/larger elements from the tail before insertion.

73. What happens if you call Deque.pop() on an empty ArrayDeque?

It throws `NoSuchElementException` because `pop()` delegates to `removeFirst()`.

74. What is the difference between pollFirst() and pollLast() in Deque?

`pollFirst()` retrieves and removes the head element; `pollLast()` retrieves and removes the tail element.

75. Can you use TreeSet with custom objects without implementing Comparable if you pass a lambda comparator?

Yes: `new TreeSet<>(Comparator.comparing(Student::getId))` works without modifying the `Student` class.

76. What is the time complexity of HashSet.clear()?

$O(C)$ where C is the internal bucket array capacity, because all bucket pointers must be set to `null`.

77. Why is Collections.unmodifiableSet() not strictly immutable?

It is an unmodifiable wrapper view; if the underlying backing set is mutated, the view reflects those changes.

78. What happens if you modify an element inside a PriorityQueue that changes its priority?

The heap becomes corrupted because the queue does not automatically re-heapify. You must remove and re-add the element.

79. Why does LinkedHashSet allow access-order sorting in its underlying map, but does not expose it publicly?

`LinkedHashSet` hardcodes its `LinkedHashMap` constructor call to insertion-order mode (`accessOrder = false`).

80. What is the return type of Set.toArray()?

`Object[]`. To obtain a typed array, use `set.toArray(new String[0])`.

Level 5: Real-World Scenario Decisions (Q81 – Q100)

81. Scenario: De-duplicating a stream of 10 million user IDs with maximum speed.

Choose: `HashSet<Long>` with initial capacity pre-allocated to $\approx 15 \times 10^6$ to prevent costly resizings ($O(1)$ lookups).

82. Scenario: Implementing an LRU (Least Recently Used) Cache.

Choose: `LinkedHashMap` with access-order set to `true` and overriding `removeEldestEntry()`.

83. Scenario: Tracking real-time top 10 trending hashtags on Twitter.

Choose: Min-Heap `PriorityQueue` of fixed size 10. The root stores the 10th largest frequency; replace root when a larger frequency arrives.

84. Scenario: Maintaining unique incoming IP addresses in the exact order they arrived.

Choose: `LinkedHashSet<String>` (Guarantees uniqueness and preserves strict insertion order).

85. Scenario: Finding closest floor and ceiling timestamps in a financial trading ledger.

Choose: `TreeSet<Long>` (Supports fast $O(\log N)$ `floor()` and `ceiling()` lookups).

86. Scenario: High-throughput task scheduling thread pool buffer.

Choose: `ArrayBlockingQueue` or `ConcurrentLinkedQueue` (Thread-safe FIFO queue implementations).

87. Scenario: Breadth-First Search (BFS) graph traversal.

Choose: `ArrayDeque<Node>` for the queue and `HashSet<Node>` for visited node tracking.

88. Scenario: Evaluating mathematical expressions (e.g., Polish notation parser).

Choose: `Deque<Integer>` `stack = new ArrayDeque<>();` (LIFO stack operations with minimal memory overhead).

89. Scenario: Sliding Window Maximum algorithm in DSA.

Choose: `ArrayDeque<Integer>` as a Monotonic Deque storing indices with elements in decreasing order ($O(N)$ total).

90. Scenario: Dijkstra's Shortest Path Algorithm.

Choose: `PriorityQueue<Pair>` ordered by edge distance for extracting minimum distance vertices in $O(\log V)$.

91. Scenario: De-duplicating student records where students with the same roll number must be considered duplicates regardless of name.

Choose: `HashSet<Student>` with `equals()` and `hashCode()` implemented using only the `rollNo` field.

92. Scenario: Implementing undo/redo buffers in a text editor.

Choose: Two `ArrayDeque<Action>` stacks (one for undo history, one for redo).

93. Scenario: Round-Robin CPU process scheduler.

Choose: `ArrayDeque<Process>` (dequeue running process from head, execute slice, re-enqueue at tail).

94. Scenario: Merging K sorted linked lists.

Choose: `PriorityQueue<ListNode>` of size K storing the current head node of each list ($O(N \log K)$).

95. Scenario: Maintaining dynamically sorted streaming stock prices with duplicate prices allowed.

Choose: `PriorityQueue` or `TreeMap<Double, Integer>` (where value is a frequency counter), because `TreeSet` discards duplicate prices!

96. Scenario: Finding median from a continuous real-time data stream.

Choose: Two Heaps — Max-Heap for lower half, Min-Heap for upper half ($O(\log N)$ insert, $O(1)$ median).

97. Scenario: Print all words from a dictionary sorted alphabetically with no duplicates.

Choose: `TreeSet<String>` (Maintains natural alphabetical lexicographical order while eliminating duplicate words).

98. Scenario: Job scheduling system where emergency jobs bypass normal tasks.

Choose: `PriorityQueue<Job>` with emergency flag weighted heavily in custom comparator.

99. Scenario: Parsing nested HTML/XML syntax tags.

Choose: `Deque<String>` `tagStack = new ArrayDeque<>();` for pushing opening tags and validating against closing tags.

100. Scenario: Deduplicating large text files where memory is constrained.

Choose: Bloom Filter (probabilistic check) followed by external merge sorting or an on-disk embedded B-Tree.

PART 6: 15 Progressively Difficult Coding Problems

Problem 1: Remove Duplicates from an Array (Easy)

```
public static int[] removeDuplicates(int[] arr) {
    Set<Integer> set = new LinkedHashSet<>();
    for (int x : arr) set.add(x);
    return set.stream().mapToInt(Integer::intValue).toArray();
}
```

Problem 2: Find Kth Largest Element in an Array (Medium)

```
public static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) minHeap.poll();
    }
    return minHeap.peek();
}
```

Problem 3: Sliding Window Maximum using Monotonic Deque (Hard)

```
public static int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>(); // Stores indices

    for (int i = 0; i < n; i++) {
        // Remove indices outside current window
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFirst();
        // Maintain monotonic decreasing order
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast();

        deque.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```

Problem 4: Implement Queue using Two Stacks (Medium)

```
public class MyQueueUsingStacks {
    private final Deque<Integer> inStack = new ArrayDeque<>();
    private final Deque<Integer> outStack = new ArrayDeque<>();

    public void push(int x) { inStack.push(x); }

    public int pop() {
        shiftStacks();
        return outStack.pop();
    }

    public int peek() {
        shiftStacks();
        return outStack.peek();
    }

    private void shiftStacks() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) outStack.push(inStack.pop());
        }
    }
}
```

PART 7: Master Revision Cheat Sheet

Collection	Underlying Structure	Ordering Guaranteed	Duplicates Allowed?	Null Allowed?	Primary Complexity	Biggest Trap to Remember
HashSet	HashMap (Hash Table)	None (Unordered)	No	Yes (1 null)	$O(1)$ avg add/contains	Must override both <code>hashCode()</code> and <code>equals()</code> .
LinkedHashSet	LinkedHashMap (Hash + List)	Strict Insertion Order	No	Yes (1 null)	$O(1)$ avg add/contains	Preserves insertion order, NOT sorted order.
TreeSet	TreeMap (Red-Black Tree)	Sorted (Natural/Comparator)	No	No (Throws NPE)	$O(\log N)$ add/contains	Duplicates detected via <code>compareTo() == 0</code> , not <code>equals()</code> .
ArrayDeque	Circular Array Buffer	FIFO / LIFO Endpoints	Yes	No (Throws NPE)	Amortized $O(1)$ head/tail	<code>null</code> is forbidden (used as empty sentinel).
PriorityQueue	Binary Heap (Array-backed)	Min-Heap Root Priority	Yes	No (Throws NPE)	$O(1)$ peek, $O(\log N)$ poll	Iteration does NOT yield sorted order.

★ The 15 Golden Rules of Java Collections

1. Always program against interfaces: `Set<T> s = new HashSet<>();`, `Deque<T> dq = new ArrayDeque<>();`.
2. If `a.equals(b) == true`, their `hashCode()` values MUST be identical.
3. Never mutate fields of an object after inserting it into a `HashSet` or `HashMap`.
4. `TreeSet` defines equality strictly through `compareTo() == 0` or `compare() == 0`.
5. Never use `java.util.Stack`; use `Deque<T> stack = new ArrayDeque<>();`.
6. Use `offer()` / `poll()` / `peek()` for safe queue operations without unexpected runtime exceptions.
7. `PriorityQueue` is a binary heap, NOT a sorted list. Use successive `poll()` calls to extract sorted data.
8. `ArrayDeque` is faster than `LinkedList` for queues and stacks due to contiguous CPU cache locality.
9. To sort custom objects in modern Java, prefer `Comparator.comparing()` lambdas over verbose anonymous classes.
10. `Set.of()` and `List.of()` create unmodifiable collections that reject `null` values immediately.
11. Sub-views (`subList`, `subSet`) are backed by their parent collections; mutating parents invalidates views.
12. `PriorityQueue` natural ordering is a Min-Heap. Use `Collections.reverseOrder()` for a Max-Heap.
13. Always pass initial capacity if size is known in advance to avoid expensive dynamic array resizings.
14. Iterators throw `ConcurrentModificationException` if the underlying collection is modified during iteration.
15. `LinkedHashSet` guarantees insertion order while maintaining $O(1)$ average hash table speed.